TECHNICAL GUIDE

Retrieval-Augmented Generation (RAG)

Complete system architecture, document flow,

implementation guide & best practices

Indexing Pipeline • Query Pipeline • Architecture • Best Practices

01

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that enhances Large Language Models by grounding their responses in real, domain-specific knowledge retrieved at query time. Instead of relying solely on what the model learned during training, RAG connects the LLM to an external knowledge base — giving it access to up-to-date, accurate, and private data.

Problem with base LLMs	How RAG solves it
Knowledge is frozen at training cutoff	Retrieves live or updated documents at query time
Cannot access private or proprietary data	Searches your own vector database of private docs
Prone to hallucination on unknown topics	Grounds every answer in retrieved source chunks
Cannot cite sources or provide references	Returns source documents alongside each answer

02

Real-World Use Cases

Use case	Knowledge source	Example query
Customer support bot	Product docs, FAQs	"What is your refund policy?"
Legal assistant	Case law, contracts	"Find precedents for IP infringement"
Medical Q&A;	Clinical guidelines	"Recommended dosage for X condition"
Enterprise search	Internal wikis, Confluence	"How do I request PTO?"
Code assistant	Internal codebase	"How does the auth module work?"

03

Indexing Pipeline

The indexing pipeline runs offline — typically once, or whenever your document base changes. It transforms raw files into a searchable vector database that the query pipeline uses at runtime.

01	**Load documents**	Read raw files: PDFs, Word docs, HTML pages, databases, plain text. Tools: PyPDFLoader, WebBaseLoader, CSVLoader.	→
02	**Split into chunks**	Break each document into overlapping text windows (300–600 chars, 50-char overlap) to preserve context across boundaries.	→
03	**Generate embeddings**	Pass each chunk through an embedding model (e.g. text-embedding-3-small) to produce a dense float vector.	→
04	**Store in vector DB**	Persist all vectors and their source metadata in a vector database: FAISS (local), Pinecone, ChromaDB, or Weaviate.	

Code example — indexing pipeline

```
from langchain.document_loaders import PyPDFLoader from langchain.text_splitter import RecursiveCharacterTextSplitter from langchain.embeddings import OpenAIEmbeddings from langchain.vectorstores import FAISS # 1. Load loader = PyPDFLoader("knowledge_base.pdf") docs = loader.load() # 2. Chunk splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50) chunks = splitter.split_documents(docs) # 3. Embed + Store embeddings = OpenAIEmbeddings() vectorstore = FAISS.from_documents(chunks, embeddings) vectorstore.save_local("faiss_index")
```

04

Query Pipeline

The query pipeline runs in real time for every user question. It converts the query into an embedding, retrieves the most relevant chunks, builds an augmented prompt, and sends it to the LLM for a grounded, source-backed answer.

01	**Receive user query**	The raw natural-language question arrives from the user interface or API endpoint.	→
02	**Embed the query**	The same embedding model used during indexing converts the query text into a float vector.	→
03	**Semantic search**	Vector DB performs cosine (or dot-product) similarity search, returning the top-k closest chunks.	→
04	**Build augmented prompt**	Retrieved chunks are injected as context before the user query in the LLM prompt template.	→
05	**LLM generates answer**	The LLM reads the context and produces a grounded, factual answer with optional source citations.	

Prompt template structure

```
System: You are a helpful assistant. Answer ONLY using the context below. Context:
[Chunk 1 — most relevant passage from vector DB] [Chunk 2 — second most relevant
passage] [Chunk 3 — third most relevant passage] User: What is the refund policy?
```

Code example — query pipeline

```
from langchain.vectorstores import FAISS from langchain.embeddings import OpenAIEmbeddings
from langchain.chat_models import ChatOpenAI from langchain.chains import RetrievalQA # Load
existing index embeddings = OpenAIEmbeddings() vectorstore = FAISS.load_local("faiss_index",
embeddings) retriever = vectorstore.as_retriever(search_kwargs={"k": 3}) # Build RAG chain llm
= ChatOpenAI(model="gpt-4", temperature=0) qa_chain = RetrievalQA.from_chain_type( llm=llm,
retriever=retriever, return_source_documents=True ) # Query result = qa_chain("What is the
refund policy?") print(result["result"]) # grounded answer print(result["source_documents"]) #
cited chunks
```

05

Key Components

Component	Best choice	Alternatives
Document loader	LangChain loaders	LlamaIndex, Unstructured.io
Text splitter	RecursiveCharacter	TokenText, SemanticChunker
Embedding model	OpenAI text-emb-3-small	BAAI/bge, Cohere Embed, HuggingFace
Vector database	Pinecone	FAISS, ChromaDB, Weaviate, Qdrant
Retriever	LangChain retriever	LlamaIndex retriever, Haystack
Reranker	Cohere Rerank	FlashRank, BGE Reranker, Cross-encoder
LLM	GPT-4 / Claude	Llama 3, Mistral, Gemini 1.5
Orchestration	LangChain	LlamaIndex, Haystack, Semantic Kernel
Evaluation	RAGAS	TruLens, DeepEval, UpTrain

06

Best Practices

01 Tune chunk size

300–600 characters works for most use cases. Too small loses context; too large dilutes similarity scores. Test with your actual documents.

02 Add chunk overlap

Set 10–20% overlap (e.g. chunk_size=500, chunk_overlap=50) so sentences at boundaries are never split mid-thought.

03 Use hybrid search

Combine dense vector search (semantic) with BM25 keyword search. Hybrid retrieval significantly improves precision on exact terms and names.

04 Rerank results

Apply a cross-encoder reranker (Cohere Rerank, FlashRank) after initial retrieval to reorder the top-k chunks by true relevance.

05 Add rich metadata

Tag chunks with source filename, creation date, section, and category. This enables pre-filtered retrieval for faster, more precise results.

06 Evaluate with RAGAS

Measure faithfulness (is the answer grounded?), answer relevance, context recall, and precision using RAGAS or TruLens on a golden dataset.

07

Full Architecture Overview

Layer	Component	Role	Runs
Ingestion	Document loader	Read and parse source files	Offline
Ingestion	Text splitter	Divide docs into overlapping chunks	Offline
Indexing	Embedding model	Convert chunks to dense vectors	Offline
Storage	Vector database	Store and index all chunk vectors	Offline
Retrieval	Query embedder	Embed the user query	Online
Retrieval	Retriever + reranker	Find and rank top-k relevant chunks	Online
Generation	Prompt builder	Assemble context + query into a prompt	Online
Generation	LLM	Generate grounded answer with citations	Online

Quick tip: Always use the same embedding model for both indexing and querying. Switching models between the two phases will produce incompatible vector spaces and drastically reduce retrieval quality.